n = 83 K600: the Bernstein–Lange gap on the rank-probe producer (a measurement)

2026-10-19

n = 83 K600: the Bernstein–Lange gap on the rank-probe producer (a measurement)

Status (2026-10-19): protocol frozen, Tier 1 run pending. The protocol and the schedule were committed before any n = 83 timing of this experiment. Nothing below is a result of it. The figures in "Power" come from a model-based prior; they size the panel and are not measurements. The local checks used Python only (see "Local checks"): the cloud-agent VM had about 2 GB of free disk, so nothing was built here and the runner builds both crates.

This is a measurement, not an index-calculus improvement. It changes no IC code and never replaces or promotes a baseline row. It measures the distance from the current n = 83 baseline to Bernstein–Lange precomputation rho (AGENTS.md, "Success criterion", "Distance to rho"), which fills the n = 83 row's "Online / BL" column in `docs/ic/BASELINES.md`.

Question

The n = 83 a = 1 K600 baseline on origin/main (1158c2152, checked at freeze) is the rank-probe producer: arms cP / kP, `koblitz-n83-rankprobe-20261019`, run 39214027289. Its row reads end-to-end cold `S` 6.390, a gap of 64.0 [53.7, 78.0] to the tuned rho rT770, and an online gap of 0.129 [0.085, 0.183]. Its "Online / BL" column reads "not measured on this row".

The only measured n = 83 BL gap is B's 235 [61, 645]. It comes from `koblitz-n83-bl-arm-20261014` (run 38689817618, PR #918, unmerged), on B's producer. That producer had a 549.8 s budget and a 1.39 s mean IC online time. Figure 1 of the ledger dropped the n = 83 BL point for that reason (lead M1 of the 2026-10-19 lead survey).

The question: on EC1N83Ce1h35c7ced6b497 (r = 8569786107849059 ≈ 2^52.93, 166 signed-Frobenius automorphisms), give a Bernstein–Lange precomputation rho the same budget as the current baseline's precomputation, measured in this job. How much faster is it online than the baseline's IC on 64 new public points? What are the end-to-end cold `S` of IC, BL and rT770, and each one's gap to rT770?

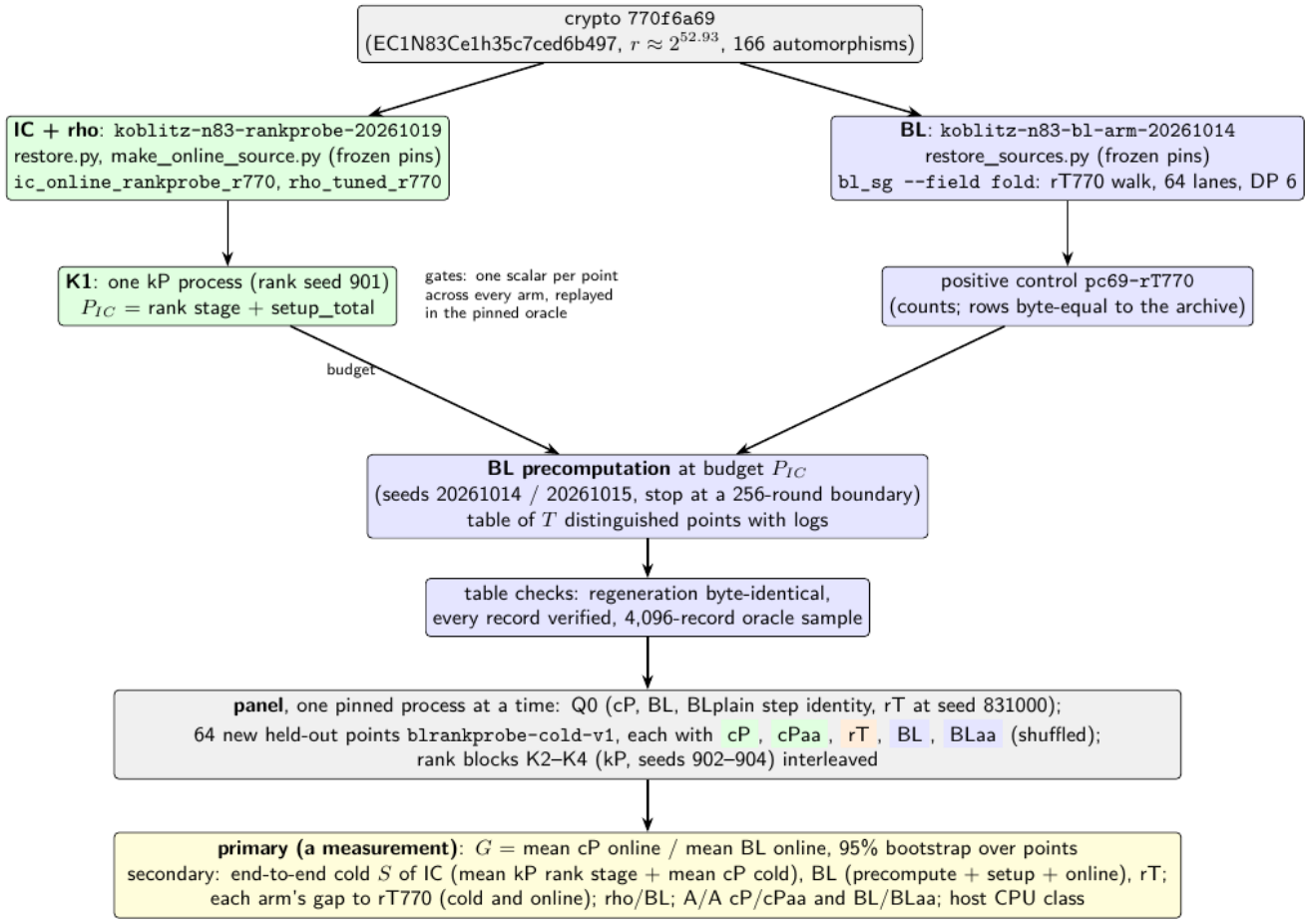


Figure 1: Design (from the frozen protocol; not a result). Source: figures/design.tex, vector: figures/design.pdf.

What is reused, unchanged

Every arm is built and wrapped by the frozen experiment it comes from. This experiment adds only the schedule, the runner, the panel, the budget hand-off and the verifier.

- **IC and rho (the baseline row's arms).** These come from `../koblitz-n83-rankprobe-20261019/`:
  - its `restore.py` restores crypto 770f6a69 and derives the rank-probe producer, checked against its own frozen `protocol.json` (`online_source_sha256d4394df3...`);
  - its `online/` crate builds `ic_online_rankprobe_r770` and `rho_tuned_r770`;
  - its `arm.py` runs arm ids `cP` (cold, cached factor-log table), `kP` (rank stage with `KIC_RANK_SOLVE=frame`, then one target) and `rT`, so its own layout, fold, index-digest, counter and log-table checks apply to every IC row.
- **BL (the BL-arm machinery, ported as that experiment defined it).** This comes from `../koblitz-n83-bl-arm-20261014/`. Its `restore_sources.py` generates `bl_sg` from crypto 770f6a69 by exact, counted edits, checked against its frozen `protocol.json` and B's. The run uses the implementation that protocol set, `rho_reference.value = rT770`: the library walk with the fold field, 64 lanes, DP 6, the tuned rho's own walk. Its tools are used unchanged:
  - `bl_table.py precompute`: jump seed 20261014, start seed 20261015, a stop at the first 256-round boundary at which its own clock has reached the budget, and 40-byte sorted records;
  - `bl_table.py check`: regeneration to the recorded round count, which must be byte-identical; every record checked by the producer (log range, distinguished, canonical, [a]G = P); a 4,096-record sample in the pinned pure-Python oracle;
  - `arm.py`: kinds `bl_sg` and `bl_plain_sg`;
  - `control.py`: the positive control.

- **The budget (the BL-arm experiment's matched cold cost).** P_IC is `timing_ms.rank_stage+timing_ms.setup_total` of one in-job IC process with the rank stage, on the execution CPU, one thread. Here that is segment K1's kP process (rank seed 901). `run_schedule.py--phasebudget` writes it as `ic_budget/precompute_receipt.json`, the file `bl_table.pyprecompute--ic-receipt` reads. As in the BL-arm experiment, three costs are left out of P_IC: the construction of the retained K600 base (427.5 ms, measured in `koblitz-n83-replay-20261017`), linear algebra and the log-table write. That can only favour the IC. Pre-run estimate: the mean of the four kP rows of run 39214027289, 163.6 s, against B's 882 s.

What differs from the BL-arm run:

- the IC producer is the current baseline, not B's;
- the budget is about 0.30 of B's;
- the panel is new;
- only the rT770 implementation runs, both in its positive control and online.

The precomputation seeds are B's. The table is built with the same walk from the same starts, so it is deterministic in the budget's round count.

Frozen Tier 1 design

One GitHub `ubuntu-24.04` runner, one pinned CPU, Tier 1 counters, and one fresh process per run.

Arm	Wrapper (arm id or kind)	Role
`cP`	rank-probe `arm.py` (`cP`, `ic_cold`)	IC cold: setup + one-target online with B's verified factor-log table; its online time is the primary's numerator
`cPaa`	rank-probe `arm.py` (`cP`)	**A/A of the IC arm**: the same arm in its own process
`kP`	rank-probe `arm.py` (`kP`, `ic_rank`)	IC rank stage to full rank, then one target; K1 gives P_IC; K1–K4 give the end-to-end cost
`rT`	rank-probe `arm.py` (`rT`, `rho_tuned`)	the tuned single-target rho rT770 (Fold/64/DP6)
`BL`	BL-arm `arm.py` (`bl_sg`)	BL online on the rT770 walk with this job's table
`BLaa`	BL-arm `arm.py` (`bl_sg`)	the BL-arm experiment's A/A: BL again, same point and seed
`BLplain`	BL-arm `arm.py` (`bl_plain_sg`)	step identity: no table, rT770's own jumps, on Q0 at seed 831000

Order.

1. Positive control `pc69-rT770`: step counts only, on all CPUs.
2. **K1** (budget phase).
3. BL precomputation at P_IC.
4. Table checks.
5. The panel phase:
   - O1: Q0 with cP (seed 7), and BL, BLplain and rT at seed 831000;
   - C1–C21, K2, C22–C42, K3, C43–C64, K4.

   Each C segment is one new point with cP, cPaa, rT, BL and BLaa, shuffled with `random.Random(2026101960+i)`. The rT, BL and BLaa seeds are 930000 + point seed: the BL-arm experiment's rule, under which BL and rho share start streams but not jump tables. K2–K4 use rank seeds 902–904. Rank seeds 901–904 are new: the stacked run used 101–102, the rank-cut run 201–204, the rank-probe run 301–304, and the scratch branches use 401–404, 501–504, 701 and 801–816.
6. The time guard is 300 minutes; the job limit is 350.

Cold panel. 64 new public points (`inputs/cold_targets.json`, role `blrankprobe-cold-v1`, built by `make_targets.py` from B's hash-to-curve law; no scalar is known for any of them). `make_targets.py` regenerates the following panels of the law and checks that no point repeats:

- **On origin/main:**
  - Q0 and B's panel;
  - `scanorder-heldout`, `combined-primary-v1`, `probepipe-primary-v1`, `statefold-cold-v1`, `tablefilter-primary-v`
    `stacked-cold-v1`, `rankcut-cold-v1` and `rankprobe-cold-v1`;
  - the 64 `eval` points of `n131-ref-walk-20261015`.
- **In the scratch worktrees** (each to 256 seeds, more than any of them uses):
  - n83-indexlift (`indexlift-cold-v1`);
  - n83-rotkey (`rotkey-cold-v1`);
  - n83-k899 (`k899-cold-v1`);
  - n83-onlineframe (`onlineframe-v1`);
  - n83-closure (`closure-cold-v1`).
- **n83-knownlog**, which had no panel at freeze. Its likely names are excluded to 256 seeds: `knownlog-cold-v1`, `known-log-cold-v1` and `knownlog-v1`.

Primary (frozen rule)

G = mean cP online_ms / mean BL online_ms over the 64 cold points. That is the online gap IC/BL, stated the way B's 235 was. Its interval is a 95% percentile bootstrap: 10,000 draws, seed 2026101990, with the points resampled and each point's (cP, BL) pair kept together. G is reported as a **measurement**.

- **Readings** (the BL-arm experiment's):
  - `bl_online_faster_than_ic` if the lower bound is above 1;
  - `ic_online_faster_than_bl` if the upper bound is below 1;
  - `not_separated` otherwise;
  - `no_result` if any gate or control fails, or any scheduled row is missing (time-guard skips included).
- **Gates.**
  - Every scheduled row is valid.
  - Every IC process passes the rank-probe wrapper's checks.
  - Per point: one scalar across every arm on the point (cP, cPaa, rT, BL, BLaa, and kP on its unit), and that scalar replays as [d]G = Q in the pinned oracle.
  - Per point: one relation and one probe count across every IC process.
  - Q0: cP gives the archived Feistel relation and scalar with 1,987,852 probes. Every Q0 arm recovers 3141592653589793.
  - Every kP factor-log table equals B's, log for log.
  - Every BL row loaded this job's table (entry count).
  - BL and BLaa take identical steps on every point.
- **Controls.**
  - **IC A/A:** the ratio of means cP / cPaa must be inside [0.97, 1.03] cold and inside [0.95, 1.05] online.
  - **BL A/A (the BL-arm experiment's):** the geometric mean of BL / BLaa online must have its 95% interval inside [0.9, 1.1].
  - **BL positive control:** `pc69-rT770` must report PASS by its own gate (amendment A1), and its `rows.jsonl` must be byte-identical to the archived run's (`3397ade0...`).
  - **BL step identity:** BLplain on Q0 at seed 831000 must take 12,592,070 steps, the same as this job's rT on Q0 at seed 831000.
  - **BL table checks:** all three must pass.
  - **BL budget:** the precompute receipt must PASS, and its budget must equal this job's P_IC.

Secondaries (none decides)

- **rho / BL online:** mean rT online over mean BL online.
- **Online gaps to rT770:** cP / rT and BL / rT.
- **End-to-end cold of each arm:**
  - IC: E2E(kP, cP) = the mean `timing_ms.rank_stage` over K1–K4 + the mean cP `cold_in_process_ms` (the baseline row's definition);

  - BL: the precompute time + the median BL process setup (curve, jumps, table read and insert) + the mean BL online time (the BL-arm experiment's definition);
  - rT: the mean of setup + online.
- `S` **of each arm.** Cold and online, in this run's rT770 step.
- **Cold gaps to rT770, each arm.** The IC's interval comes from a bootstrap over blocks, attempts within blocks, and points. Cold IC / BL is also reported; it is about 1 by construction.
- **The matched fraction:** P_IC / (E2E(kP, cP) − mean cP online).
- **Descriptives:** medians, the number of rows on which BL was faster, Q0 per arm, and which collision solved each BL row.
- **The BL model check:** mean BL steps over the frozen model at this table's covered count.
- **The table:** T, α and the duplicate walk ends.
- **The host CPU record** (next section).

Host CPU record (lead M5)

`scripts/host_cpu_record.py` is a new, standalone helper; frozen experiments keep the `isolated_bench.py` they pinned. It records:

- **The CPU:** the execution CPU's model name, vendor, family, model, stepping, microcode and MHz.
- **The flags this work depends on:** `pclmulqdq`, `vpclmulqdq`, `gfni`, `avx`, `avx2`, `bmi1`, `bmi2`, `adx`, `aes`, `vaes`, `sha_ni`, `popcnt`, `fma`, every listed `avx512*` flag and `hypervisor`, as true or false. Every `avx512*` flag present is also listed.
- **For exact comparison:** the full sorted flag list and its SHA-256; the `bugs` line; and whether every logical CPU reports the same model, microcode and flags.
- `host_class`**:** for example `AMDEPYC776364-CoreProcessor|avx512=ngfni=nvpclmulqdq=yavx2=ybmi2=y`.
- **Caches:** every cache index of the execution CPU (level, type, size, ways, line size, sets, shared CPUs).
- **Topology:** CPUs, cores and threads, and the process affinity.
- **The kernel:** `uname`, `/proc/version`, the boot command line and the transparent-huge-page settings.
- `lscpu -J`, when the tool is present.

Fields that the host does not expose are `null`, never guessed. No timing is read.

New experiments call it as follows:

- `python3scripts/host_cpu_record.py--merge-intohost-probe.json` adds `cpu_record` to a `probe-host` file;
- `--out host-cpu.json` also writes the record on its own;
- from Python, `host_cpu_record.record(cpu)`.

This workflow does all three. `allocation.json` carries `observed_host_class` and `observed_microcode`, and `run_schedule.py` stores the record in each phase's `runner_start_*.json`. On the cloud-agent VM it read `Intel(R)Xeon(R)Processor@2.10GHz|avx512=ygfni=yvpclmulqdq=yavx2=ybmi2=y`, microcode `0x1`, with L1d 48K, L1i 32K, L2 2048K and L3 266240K.

Power (`power_check.py`, `analysis/power.json`)

Run before freezing. The primary is a measurement, so the panel is sized for precision, not for a test against 1.

The prior is `inputs/power_prior.json`, built by `make_power_prior.py`:

- **IC online:** the 20 fresh cP rows of run 39214027289 (mean 310 ms, coefficient of variation 0.85).
- **BL online:** the 20 BL rows of run 38689817618, rescaled to this budget by the BL arm's frozen model. The scale factor is model steps at the covered count 3.03e9 × 163.6 / 549.8, over model steps at B's covered count: 2.90. BL's coefficient of variation is about 1.04.

The simulation uses 400 simulated panels per size, each with a 1,000-draw bootstrap of G.

The rule: COUNT is the smallest n in {20, 32, 48, 64, 80, 96} whose median width ratio (upper / lower) of the 95% interval is at most 2.0.

n	median upper/lower	90th percentile	P(lower > 1)	simulated median G
20	3.22	4.03	1.00	16.9
32	2.54	2.89	1.00	16.7
48	2.14	2.34	1.00	17.0
64	**1.95**	2.09	1.00	16.9
80	1.81	1.92	1.00	17.1
96	1.72	1.80	1.00	16.8

So the panel has 64 points.

Sensitivity. With BL steps scaled by 0.5 or 2, the width ratio is unchanged: 1.95 and 1.94 at n = 64. The simulated median G moves to 34 or 8.4. In every cell P(lower > 1) = 1 and P(upper < 235) = 1. If the model holds, then, the measured gap on the current producer would be resolved from 1 and from B's 235.

These are predictions, not results. The expected G of about 17 is the prior's: IC online is about 4.5× faster than B's producer's, and the BL budget is about 0.30 of B's.

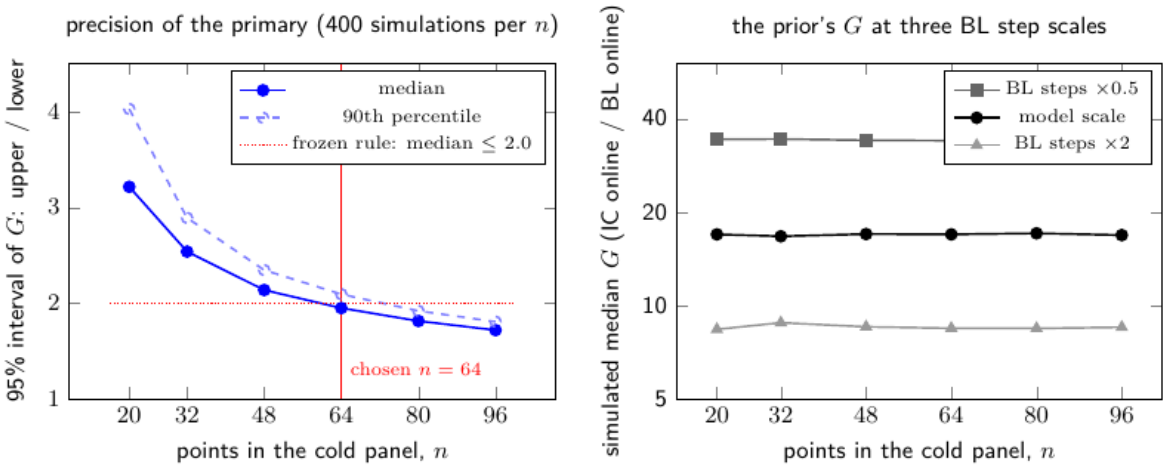


Figure 2: Sizing simulation (model-based prior; not a result). Source: figures/power.tex written by figures/make_figures.py from `analysis/power.json`, data figures/power.dat, vector: figures/power.pdf.

Runtime estimate. About 2.3 h, and up to about 3 h on a slower runner:

Stage	Estimate
build and tests (two crates)	about 25 min
positive control	about 2 min
K1	about 170 s
precomputation	about P_IC
table checks	about 6 min (B's 48 M-record table took 17 min)
each cold segment (cP and cPaa about 23 s each, rT about 3 s, BL and BLaa about 4 s each with the table load)	about 60 s
K2–K4	about 170 s each

Local checks (cloud-agent VM, Python only; no timing)

Nothing was built: the VM had about 2 GB of free disk, shared with agents that were building.

- **Restore and pins.** Both restores ran with `--crypto-repo` on a local crypto clone: the rank-probe restore and the BL-arm `restore_sources.py`, each passing its own frozen pins. This experiment's `restore.py` then checked every local and sibling digest against `protocol.json`.
- **Panel and schedule.** `make_targets.py --check` regenerated the 64 points and confirmed them disjoint. `make_schedule.py --check` reproduced `schedule.json`: 328 runs, K1 alone in the budget phase.
- **Wrapper command lines.** Every arm's command line was run once against stub binaries that exit 1. Both frozen wrappers parsed every argument and reported the expected failure, `verified=0`. `run_schedule.py` ran both phases on stubs. The budget phase wrote a FAIL budget receipt and exited

  3. The panel phase checked a stub table receipt and appended to the same ledger. `verify_receipt.py--correctness-only` read the result.

- **The verifier on synthetic receipts.** `smoke/synthetic_check.py`, results in `smoke/synthetic_check.json`, runs six scenarios. The synthetic times are inputs, not measurements. Every cold point is replaced by [d]G with d known, so that the oracle replay can pass:

scenario	reading	failed check
planted gap 20	`bl_online_faster_than_ic`, G = 20	none
no gap	`not_separated`	none
one BL scalar wrong	`no_result`	`one_scalar_per_point_every_arm` and `oracle_replay`
one BLaa step count wrong	`no_result`	`bl_and_blaa_steps_identical`
BL budget ≠ P_IC	`no_result`	`bl_precompute_pass_at_this_jobs_P_IC`
control rows altered	`no_result`	`bl_positive_control_rows_equal_archived`

  Every scenario returned the reading it was built to produce. The secondary block (end-to-end `S`, the gaps, the model check, the matched fraction) was also computed on the planted receipt without error.

- **The host helper** ran on the VM, both on its own and with `--merge-into`.

Claim classification (controlled-experiment skill)

- **Exact computation:**
  - every recovered scalar and its oracle replay;
  - the table's records;
  - step identity and BL/BLaa step identity;
  - the IC relation and probe identities;
  - the factor-log tables.
- **Comparative (measurement):** G, rho/BL, the cold `S` and the gaps, read by the frozen rule with their intervals. None exists yet.
- **Heuristic estimate:** the power prior, the BL model and the runtime estimate.
- **Out of scope:**
  - any claim of an IC improvement or of beating rho;
  - ECC2K-130, E_0 over GF(2^83), or either method's speed in general;
  - any wall time from the VM.

The result is a single-runner, instance-scoped Tier 1 figure. The host class is recorded so that it can be compared with the rank-probe run (AMD EPYC 7763) and B's (AMD EPYC 9V45).

Workflow and receipt

`.github/workflows/n83-bl-rankprobe.yml` runs on a push to `claude/n83-bl-rankprobe` or by `workflow_dispatch`. Its path filter excludes:

- run_github_*/**;
- README.*;
- figures/**;
- smoke/**;
- receipts/**.

The receipt is $RUNNER_TEMP/n83-bl-rankprobe/. It holds:

- allocation.json, host-probe.json (with cpu_record), host-cpu.json;
- restore.json, frozen.sha256, rustc.txt, the cargo test logs, binaries.sha256, log-table-check.json;
- control/;
- ic_budget/precompute_receipt.json;
- bl/, with the precompute receipt, the table checks and the regeneration's stdout;
- panel/: runs.jsonl, runs/*, and runner_start_*/runner_end_* of both phases;
- result.json and result_table.md.

table.bin is uploaded as a separate artifact.

Files

File	Role
protocol.json	the frozen design, with the digest of every executed local and sibling file (make_protocol.py)
schedule.json	the frozen run order (make_schedule.py; --check on the runner)
inputs/cold_targets.json	the 64 held-out points (make_targets.py; --check on the runner)
inputs/archived_identities.json	Q0's IC identity, the step identity, the positive control's rows digest, B's table digest
inputs/power_prior.json, analysis/power.json	the prior and the sizing (make_power_prior.py, power_check.py)
restore.py	runs both frozen restores and checks every pin
run_schedule.py	the budget and panel phases, one pinned process per run
verify_receipt.py	gates, controls, the primary and the secondaries
smoke/synthetic_check.py, smoke/synthetic_check.json	the synthetic verifier check
figures/	the design diagram and the sizing figure (TeX sources, PDF, PNG)
README.pdf	this report: pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true
../../scripts/host_cpu_record.py	the M5 host CPU record (shared helper)